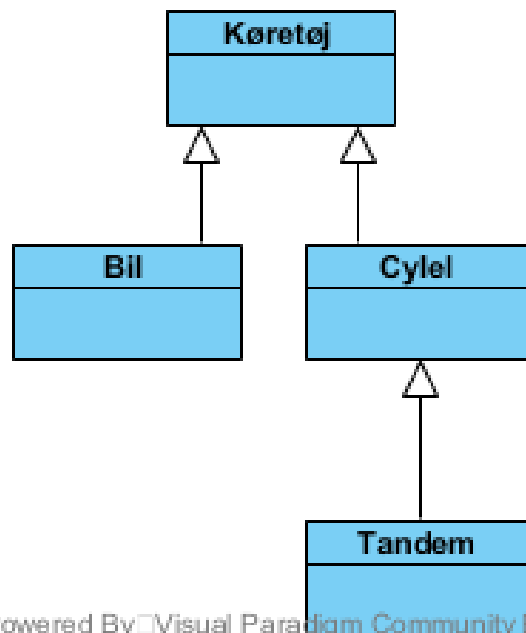


Programmering

Specialisering
Nedarvning
Polymorfi

Specialisering

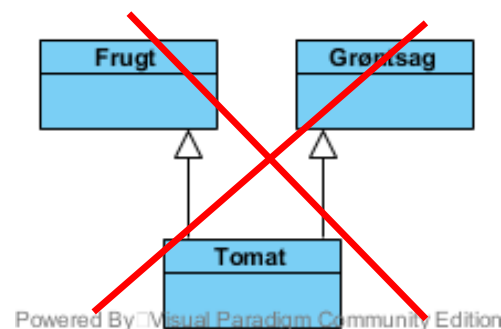
- Specialisering er det modsatte af generalisering
- Begge abstraktionsmekanismer bruges til at forstå og organisere begreber i "virkeligheden"
- Programmer skal modellere "virkeligheden"
- Derfor vigtigt at programmeringssproget understøtter disse abstraktionsmekanismer



En bil **er et** køretøj
 En cykel **er et** køretøj
 En tandem **er en** cykel
 En tandem **er et** køretøj

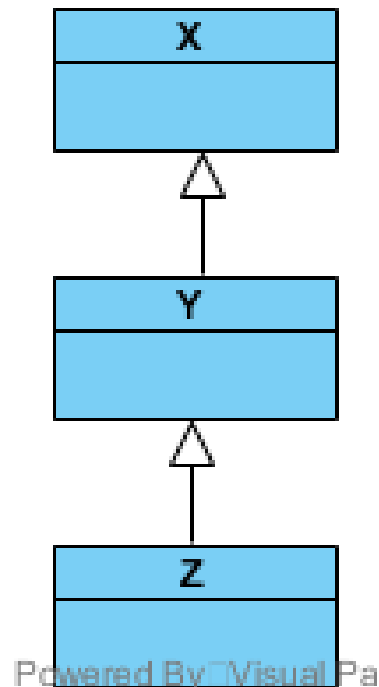
Specialisering (fortsat)

- Objektorienteret programmeringssprog understøtter specialisering - men ikke generalisering
 - C++ understøtter multipel arv
 - Java, C# og de fleste andre objektorienterede sprog understøtter enkelt arv
- UML understøtter både specialisering og generalisering



Multipel arv understøttes
ikke i Java

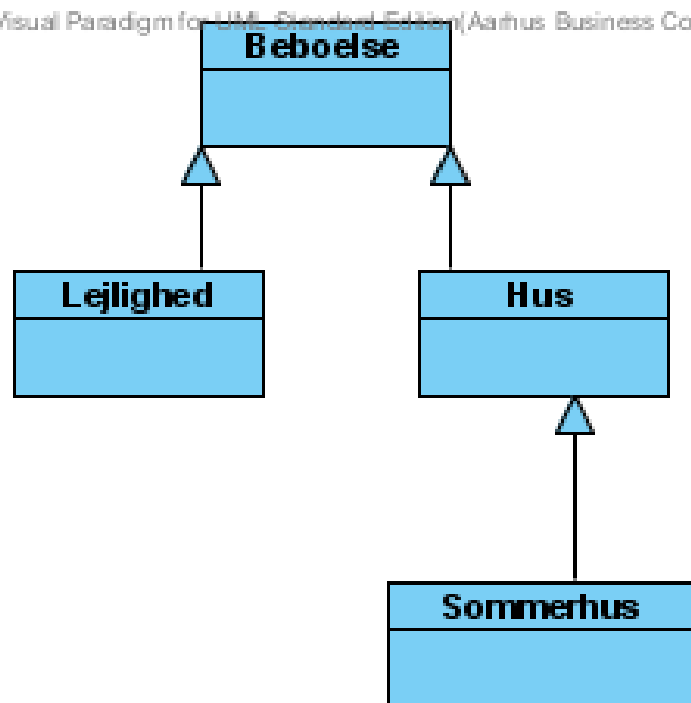
Sub- og superklasser



- Hvis klassen Y specialiserer klassen X, kaldes X for **superklassen** og Y for **subklassen**
- Hvis klassen Z yderligere specialiserer klassen Y, er X indirekte også en superklasse Z og Z er indirekte en subklasse af X
- En klasse kan have **mange subklasser**, men kun en direkte superklasse
- Klasser danner på den måde et **specialiserings hierarki**

Klassehierarki

Visual Paradigm for UML Standard Edition (Aarhus Business College)



Arv

- En subklasse arver alle metoder og attributter i superklassen
- Arv er en form for genbrug
- `private` forhindrer, at de er synlige i subklassen
- Attributter skal altid være `private`

Udvider

- Attributter og metoder, defineret i en subklasse, udvider den ift. de attributter og metoder, der arves fra superklassen
- I Java bruges nøgleordet `extends` til at angive, at en klasse er en subklasse af en klasse
- `class Y extends X{...}`

```
public class Beboelse {
    private String adresse;
    private int kvm;

    public Beboelse(String adresse, int kvm) {
        this.adresse = adresse;
        this.kvm = kvm;
    }

    public String getAdresse() {
        return adresse;
    }

    public int getKvm() {
        return kvm;
    }
}
```

Super klassen kodes som vanligt og har ikke kendskab til subklasser

Udvider

```
public class Hus extends Beboelse {
    private int grund;

    public Hus(String adresse, int kvm, int grund) {
        super(adresse, kvm);
        this.grund = grund;
    }

    public int getGrund() {
        return grund;
    }
}
```

```
public class Sommerhus extends Hus {
    private boolean beboet;

    public Sommerhus(String adresse, int kvm, int grund, boolean beboet) {
        super(adresse, kvm, grund);
        this.beboet = beboet;
    }

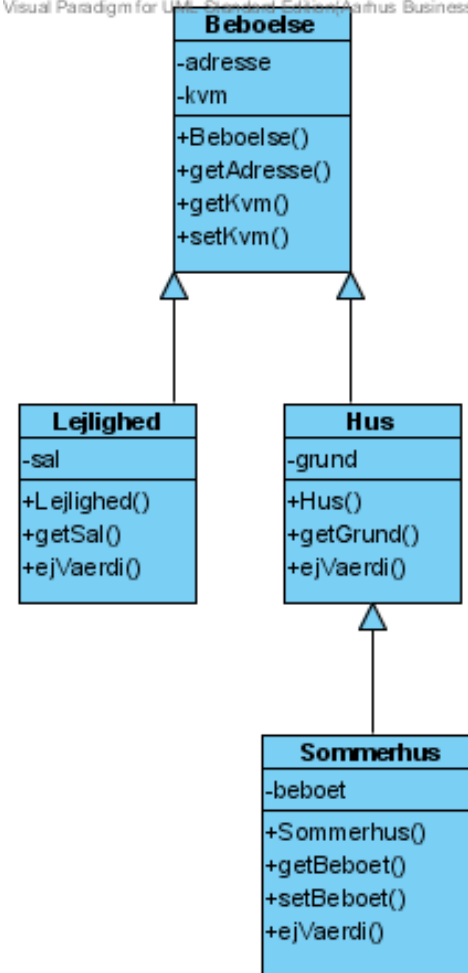
    public boolean isBeboet() {
        return beboet;
    }
}
```


Constructor og super

- constructor's arves ikke
- En superklasses constructor kan kaldes med `super(...)` i første linie af subklassens constructor
- Hver klarer sit – dvs. princippet er uddelegering

Override metode

Visual Paradigm for UML - Product Edition - Aarhus Business Col



Override

- En metode siges at override en arvet metode, hvis de har samme signatur
- Den arvede metode kan tilgås med nøgleordet super

Metoden erklæret i Hus

```
public int ejVaerdi() {
    return getKvm() * 290 + grund * 40;
}
```

Metoden erklæret i
Sommerhus

```
@Override
public int ejVaerdi() {
    return super.ejVaerdi() - 24;
}
```

Polymorfi

```
public static void main(String[] args) {

    Hus[] huse = new Hus[5];
    huse[0] = new Hus("Vandet 2", 110, 1500);
    huse[1] = new Sommerhus("Vandet 4", 66, 1000, false);
    huse[2] = new Hus("Havet 1", 187, 2000);
    huse[3] = new Sommerhus("Havet 2", 98, 900, true);
    huse[4] = new Hus("Havet 3", 100, 600);

    int sum = 0;
    for (Hus hus: huse) {
        sum = sum + hus.ejVaerdi();
    }
}
```

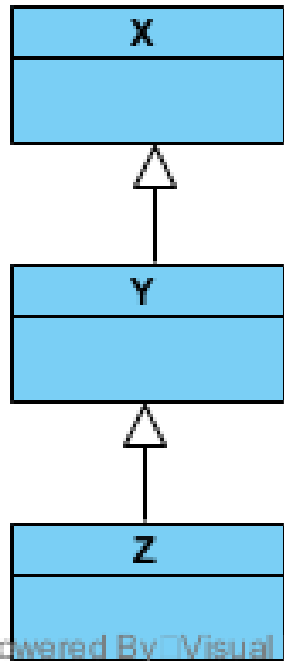
Hvilken ejVaerdi() –metode kaldes?

Cast

- En reference (attribut, variable, etc.) til et objekt kan typekonverteres (cast'es) til en subklasse af referencens type
- Hvis objektet der refereres, ikke er af den pågældende type, gives der en runtimefejl
- Anvend instanceof til at lave sikker typekonvertering

```
public static void main(String[] args) {
    Hus h = new Sommerhus("Strandvej 2", 70, 2000, true);
    Sommerhus s = new Sommerhus("Søskrænten 12", 90, 1980, false);
    h = s;
    h.getGrund();
    // h.isBeboet(); // Den erklærede type har ikke metoden isBeboet()
    if (h instanceof Sommerhus) {
        s = (Sommerhus) h; // Anvend kun dette i GUI ellers lav polymorf
        s.isBeboet();
    }
}
```

Substitutionsprincippet



Powered By Visual Pa

- Variable, attributter, parametre og returværdier kan referere objekter af en subklasse til deres egen erklærede type "Substitutionsprincippet"
- `X x = new X(); x = new Y(); x = new Z();`
- Muliggør generel kode, hvor objekter af forskellige subklasser behandles ens – dvs. samme metodekald
- Den rigtige implementation vælges automatisk – dvs. den rigtige override metode

Object

- Alle klasser arver fra klassen Object
- Alle klasser arver dermed metoderne
 - toString
 - equals
 - m.fl.